

Corruption-Aware Image Restoration with Autoencoders, Hard and Soft Expert Routing, and Style-Conditioned Face-to-Sketch Synthesis

Muhammad Ali (i230008)

Generative AI — Assignment 1, Section B

m.alenauman@gmail.com — Repository: <https://github.com/Ali-prog-spec/Restoration-Sketch-studio>

Demo video: <https://youtu.be/f5fjrYPDRv8>

Abstract—We design, train, evaluate and deploy four generative systems. Tasks 1–3 restore Oxford-IIIT Pet images corrupted at runtime by salt-and-pepper noise, Gaussian blur or rectangular occlusion: (1) a single convolutional denoising autoencoder with a genuine bottleneck, (2) a corruption classifier that hard-routes inputs to three specialist autoencoders (with an identity bypass for clean inputs), and (3) a jointly fine-tuned soft mixture-of-experts initialised from Task 2. Task 4 is a pix2pix-style conditional GAN that translates FS2K face photographs into sketches of three styles through a learned style embedding used by both generator and discriminator. All hyperparameters are tuned with Optuna, all runs are tracked with MLflow, all deployed models are exported to ONNX and verified numerically, and the four systems are served by one FastAPI + React application in Docker. On the official Oxford-IIIT Pet test set (36,690 corrupted images), the mean SSIM rises from 0.672 for the corrupted inputs to 0.786 (universal autoencoder), 0.802 (hard routing, classifier accuracy 99.42%) and 0.838 (soft mixture of experts). The face-to-sketch generator reaches a test L1 of 0.111 and SSIM of 0.462, and changing only the style condition changes the generated sketch by 0.10–0.12 mean absolute intensity.

Index Terms—denoising autoencoder, mixture of experts, routing, conditional GAN, pix2pix, Optuna, ONNX

I. Introduction

Real images are often degraded in ways that are not known in advance: sensor noise produces isolated black and white pixels, defocus produces blur, and occluders hide whole regions. A blind restoration model must remove whichever corruption is present without being told which one it is. This raises a design question that we study experimentally: is it better to train one universal model, to first recognise the corruption and send the image to a specialist (hard routing), or to let a learned gate mix several specialists continuously (soft routing)? Hard routing is interpretable and lets every expert specialise, but a single classification error sends an image to the wrong expert; soft routing is differentiable and can hedge between experts when the input is ambiguous or contains more than one corruption.

The second problem is paired image-to-image translation with a controllable output: converting a face photograph into a sketch in one of three artist styles. Here the question is whether a single conditional GAN can learn several styles through a learned style embedding, and

whether that embedding genuinely influences the output rather than serving as a label.

Our contributions are: (i) a reproducible corruption pipeline with runtime training corruptions and deterministic validation/test manifests; (ii) a universal denoising autoencoder with a genuine bottleneck, including an investigation of limited skip connections; (iii) a hard-routed system evaluated with oracle and predicted routing; (iv) a soft mixture of experts initialised from Task 2 and fine-tuned jointly, with a routing analysis; (v) a style-conditioned pix2pix model on FS2K; and (vi) one containerised web application that serves all four systems from ONNX models.

II. Related Work

Denoising autoencoders. Vincent et al. [1], [2] introduced denoising autoencoders that learn robust features by reconstructing clean inputs from corrupted ones. Convolutional encoder–decoders with symmetric skip connections [3] and residual denoisers that predict the noise [4] dominate restoration benchmarks, but their skip and residual paths carry high-resolution input information around the bottleneck, which this assignment explicitly forbids. Zhao et al. [5] showed that L2 correlates poorly with perceived quality and that mixtures of L1 and (MS-)SSIM losses work better; SSIM is defined in [6] and its relation to PSNR is discussed in [7]. Inpainting large holes is studied by context encoders [8]. Bilinear upsampling followed by convolution avoids the checkerboard artefacts of transposed convolutions [9].

Mixture of experts. Jacobs et al. [10] proposed adaptive mixtures of local experts with a softmax gate. Shazeer et al. [11] scaled sparsely gated MoE layers and introduced an importance (CV^2) loss against routing collapse; Switch Transformers [12] and GShard [13] use related load-balancing losses. Temperature-scaled softmax is used in knowledge distillation [14]; confidence penalties based on entropy regularise output distributions [15].

Conditional GANs. GANs [16] and conditional GANs [17] learn generators through an adversarial game; DCGAN [18] and [19] introduced practical stabilisation techniques. Pix2pix [20] combines a U-Net generator [21]

with a PatchGAN discriminator and an L1 term. Conditions can be injected by concatenating label maps [22], by conditional normalisation [23], [24] or through a projection discriminator [25]. Image-generation quality is often measured with LPIPS [26] or FID [27].

Hyperparameter optimisation. Optuna [28] provides define-by-run search with the TPE sampler [29] and early-stopping pruners related to successive halving [30], [31].

III. Dataset Preparation

A. Oxford-IIIT Pet (Tasks 1–3)

The official trainval list (3,680 images) [32] is split 80/20 with seed 42 into 2,944 training and 736 validation images; the 3,669 official test images are used only for the final evaluation. Images are converted to RGB and resized to 128×128 , and the same split is used by Tasks 1–3. Because the official server limited each connection to about 150 KB/s, the images were downloaded from a public mirror and verified against the official source: the mirror’s splits equal the official trainval.txt/test.txt, and all 1,145 images recovered from a partial official download are byte-identical (MD5).

B. Corruption configuration

Table I lists the corruptions. During training a new condition and severity are sampled every time an image is loaded, and a balanced batch sampler gives every batch exactly $B/4$ images of each condition. Salt-and-pepper corrupts whole pixels; blur uses reflect padding; for occlusion the coverage is the union of the rectangles, so overlapping rectangles cannot under-cover. Validation (2,944 records, four conditions per image) and test (36,690 records, clean plus three corruptions at three levels per image) corruptions are stored once in deterministic manifests that hold the type, level, seed, probability, kernel, σ , rectangle coordinates and coverage; a unit test confirms that reloading a manifest reproduces identical images. The realised test occlusion coverage is 10.00% (range 9.87–10.16%), 19.83% (18.51–20.30%) and 34.55% (33.50–35.42%) for the three levels.

C. FS2K (Task 4)

The official split (1,058 train / 1,046 test) [33] is used; 15% of the training portion is held out for validation, stratified by style with seed 42, giving 899/159 images. Photos and sketches are resized to 128×128 ; a paired augmentation (resize to 143, identical random crop and horizontal flip) keeps every photograph aligned with its sketch, which a unit test verifies. The sketches are stored as RGB but all three channels are identical, so the generator outputs one channel. Two properties of FS2K matter for the analysis: the official test set is style-imbalanced (Style 1: 619, Style 2: 381, Style 3: 46 images, whereas training is balanced), and style is confounded with the photo source (all photo2 images are Style 3, while photo1 contains almost only Styles 1 and 2).

TABLE I
Corruption configuration (assignment specification).

Condition	Training (runtime, uniform)	Test levels (low / med / high)
Clean	original image	–
Salt-and-pepper	$p \sim U(0.02, 0.15)$, black/white 50/50	$p = 0.03/0.08/0.15$
Gaussian blur	$k \in \{3, 5, 7\}$, $\sigma \sim U(0.5, 2.5)$	$(3, 0.7)/(5, 1.5)/(7, 2.5)$
Occlusion	1–3 black rectangles, union 10–35%	$\approx 10/20/35\%$, 1/2/3 rect.

IV. Experimental Setup

All models are implemented in PyTorch 2.14 (CUDA 12.6) and trained on one NVIDIA RTX 2060 (6 GB) with an Intel i7-9750H CPU, Python 3.11.9 and Optuna 5.0. Seed 42 is used for all splits, samplers and initialisations. Tasks 1–3 use mixed-precision training; the GAN is trained in full precision for stability. Every script supports SMOKE (tiny subset, used to test the whole pipeline), DEV and FINAL modes; only FINAL results appear in this report. Every Optuna study uses the TPE sampler (seed 42), median pruning and SQLite storage, and its first trial is the configuration suggested by the assignment, so each study directly compares the suggested baseline with the tuned result. All runs (hyperparameters, per-epoch losses and metrics, figures, checkpoints and the best Optuna configuration) are logged with MLflow.

For the restoration tasks the model-selection objective is $J = 0.5 \text{ L1} + 0.5 (1 - \text{SSIM})$ on the validation manifest (lower is better). It combines reconstruction error and structural similarity, as the assignment requires, but is deliberately independent of the trainable loss weight α : if trials were compared by their own training loss, Optuna could lower the objective simply by changing α . We report L1, SSIM and, as an additional standard restoration metric, PSNR [7], always together with the metrics of the corrupted input so that the gain of restoration is visible.

V. Task 1: Universal Denoising Autoencoder

A. Methodology and architecture

The universal model (Fig. 1) is a convolutional autoencoder without skip connections. Each encoder stage halves the resolution with a stride-2 convolution followed by a second convolution (BatchNorm [34], LeakyReLU); a 1×1 convolution then compresses the features into the latent code z , and a mirrored decoder of bilinear upsampling and convolutions reconstructs the RGB image through a sigmoid. Optuna chooses the bottleneck shape from five options ($8 \times 8 \times \{16, 32, 64\}$, $16 \times 16 \times \{8, 16\}$), i.e. 1,024–4,096 latent values for a 49,152-value input, a compression of 12–48 $\times$. Every option is a genuine bottleneck, so no trial can improve by removing it. The selected model uses three encoder stages with $C = 48, 96, 192$ channels and a $16 \times 16 \times 8$ latent (2,048 values, 24 $\times$ compression; 1.01 M parameters). A unit test shows that, without skips, the decoder output depends only on z .

To investigate limited skip connections we trained the same configuration with one skip at the coarsest decoder

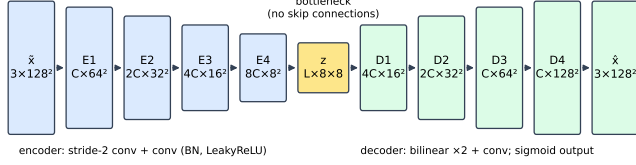


Fig. 1. Convolutional DAE. No skip connections: all information passes through the latent z .

TABLE II

Task 1 Optuna study task1_universal_dae_final: 30 trials (complete 20, pruned 10); 15 epochs per trial; best objective 0.1445 (trial 29).

Parameter	Type	Search range	Best
lr	float	[0.0001, 0.003] (log)	0.001912
batch_size	categorical	{16, 32, 64}	16
bottleneck	categorical	{8×8×16, 8×8×32, 8×8×64, 16×16×8, 16×16×16}	16×16×8
base_channels	categorical	{16, 32, 48}	48
dropout	float	[0, 0.3]	0.006019
alpha	float	[0.5, 0.95]	0.5306

level (16×16), squeezed to 4 channels by a 1×1 convolution; there is no skip at higher resolutions and no input-output residual.

B. Loss

$\mathcal{L}_{DAE} = \alpha \mathcal{L}_1(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x}))$, with SSIM (11×11 Gaussian window, $\sigma = 1.5$) implemented in PyTorch and checked against scikit-image to within $2 \cdot 10^{-3}$.

C. Optuna search

The study ran 30 trials of 15 epochs (20 completed, 10 pruned). The assignment baseline (trial 0: $\alpha = 0.8$, learning rate 10^{-3} , batch 32, $8 \times 8 \times 16$ bottleneck) reached $J = 0.250$; the best trial (29) reached $J = 0.145$, a 42% reduction. The fANOVA importances (Fig. 2) attribute most of the variance to dropout (0.62) and the learning rate (0.32): the best trials use almost no dropout (0.006), because dropout on the latent code removes information the decoder needs. The selected $\alpha = 0.53$ is lower than the suggested 0.8, i.e. structural similarity receives more weight. The selected configuration was retrained for 80 epochs (best validation epoch 79: SSIM 0.798, PSNR 25.53 dB).

D. Results

The training and validation curves (Fig. 3) decrease together without a widening gap, so the model does not overfit. On the test set the mean SSIM rises from 0.672 (inputs) to 0.786 and the mean L1 falls from 0.050 to 0.039 (Tables III–IV). The benefit depends strongly on the corruption. Salt-and-pepper is removed almost completely: SSIM rises from 0.382 to 0.825, and the output quality hardly depends on the noise level (0.830/0.826/0.818 for $p = 0.03/0.08/0.15$), because isolated extreme pixels are easy to detect and the bottleneck cannot represent them.

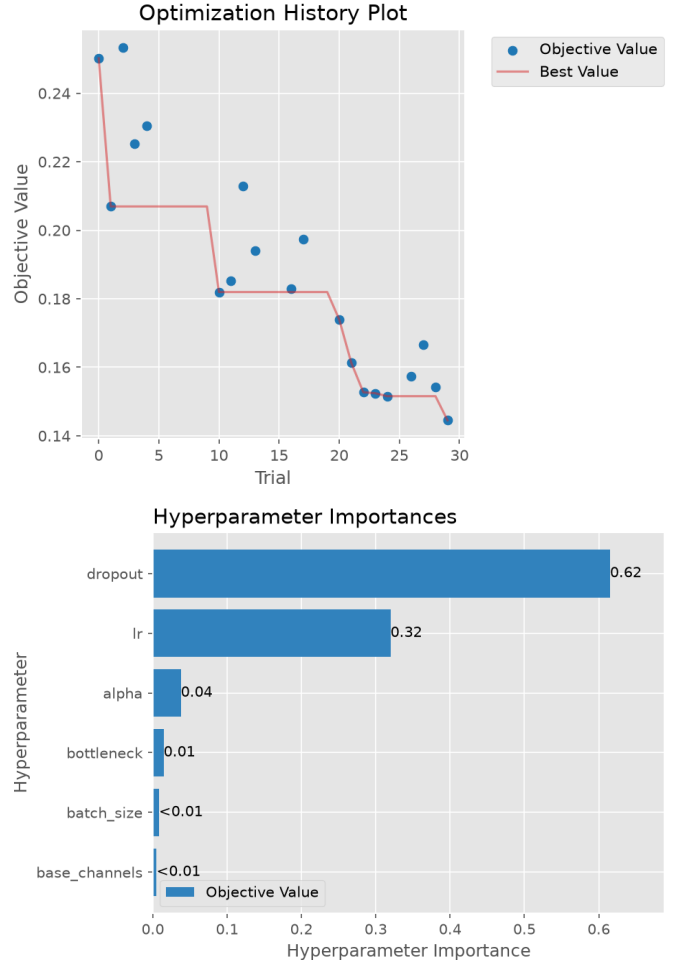


Fig. 2. Task 1 Optuna optimisation history and parameter importance.

TABLE III

Task 1 test results per condition (input_* = corrupted input vs. clean target).

Condition	l1	psnr	ssim	input_psnr	input_ssim	n
clean	0.0328	26.70	0.8310	100.00	1.0000	3669
salt_pepper	0.0331	26.60	0.8246	16.38	0.3821	11007
blur	0.0343	26.29	0.7976	27.82	0.8136	11007
occlusion	0.0507	21.71	0.7220	13.61	0.7126	11007
overall	0.0387	25.05	0.7864	27.35	0.6725	36690

Occlusion is improved moderately overall (0.713→0.722): large holes gain most (0.546→0.653 at 35% coverage), but at 10% coverage the output is worse than the input (0.862→0.781), because re-encoding the whole image costs more detail than filling the small hole gains. For blur the model helps at medium and high severity (0.806→0.820, 0.692→0.746); for mild blur (input SSIM 0.944) and clean inputs (output SSIM 0.831) the output is worse than the input, because the $24 \times$ bottleneck cannot transmit fine texture. Fig. 4 shows this crossover: restoration gains grow with severity, while near-clean inputs lose detail. Averaged

TABLE IV
Task 1 test results per condition and severity.

Condition	l1	psnr	ssim	input_psnr	input_ssime	n
clean / none	0.0328	26.70	0.8310	100.00	1.0000	3669
salt_pepper / low	0.0328	26.70	0.8300	20.13	0.6025	3669
salt_pepper / medium	0.0330	26.64	0.8261	15.88	0.3397	3669
salt_pepper / high	0.0337	26.46	0.8177	13.14	0.2041	3669
blur / low	0.0323	26.88	0.8271	32.34	0.9436	3669
blur / medium	0.0327	26.72	0.8196	26.77	0.8057	3669
blur / high	0.0380	25.27	0.7459	24.35	0.6916	3669
occlusion / low	0.0405	23.70	0.7813	16.69	0.8619	3669
occlusion / medium	0.0487	21.81	0.7315	13.36	0.7301	3669
occlusion / high	0.0628	19.61	0.6531	10.79	0.5458	3669

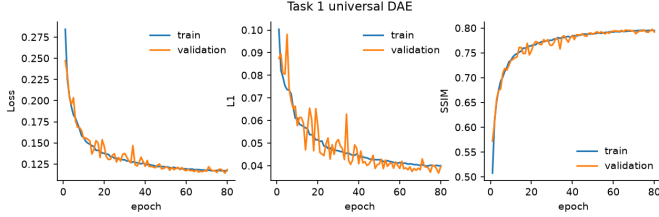


Fig. 3. Task 1 training and validation curves.

over corruptions, the output SSIM is 0.813/0.792/0.739 for low/medium/high severity, while the input SSIM falls much faster (0.803/0.625/0.481). The twelve examples in Fig. 5 show the same pattern: noise is removed and edges are kept, occlusions are replaced by smooth colour that roughly continues the surroundings, and the error maps concentrate at fine textures (fur, grass) and inside former occlusions.

Limited skips. The ablation with one narrow skip at 16×16 improves every condition (overall SSIM 0.839 vs. 0.786; clean 0.898 vs. 0.831; occlusion 0.771 vs. 0.722). Even a 4-channel skip at a coarse resolution carries enough spatial detail to sharpen the outputs. Because the assignment requires all information to pass through a genuine bottleneck, the deployed model is the no-skip version; the ablation quantifies the fidelity this constraint costs.

E. Failure analysis

Fig. 6 shows seven automatically selected failures. (1) A spotted cat on a black background with 35% occlusion (SSIM 0.723→0.299): the model fills the black background with the grey-green average colour it learned for pet backgrounds, so most of the error is outside the cat; the model has no notion of which pixels are visible. (2) A dog in dense flowers under strong blur (0.335→0.364): the texture is beyond what the bottleneck can recover, and the output is only slightly sharper. (3) A beagle in a meadow with 15% salt-and-pepper (0.372→0.504): the noise is removed but so is the grass texture. (4–7) Clean images with high-frequency patterns (striped blanket, grass, fence) lose the pattern (SSIM 1.0→0.54–0.56): the universal model is never told that the input is

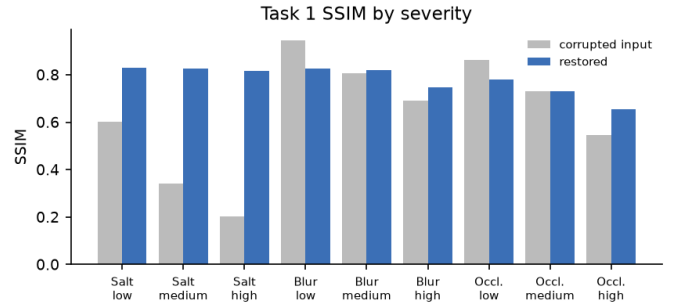
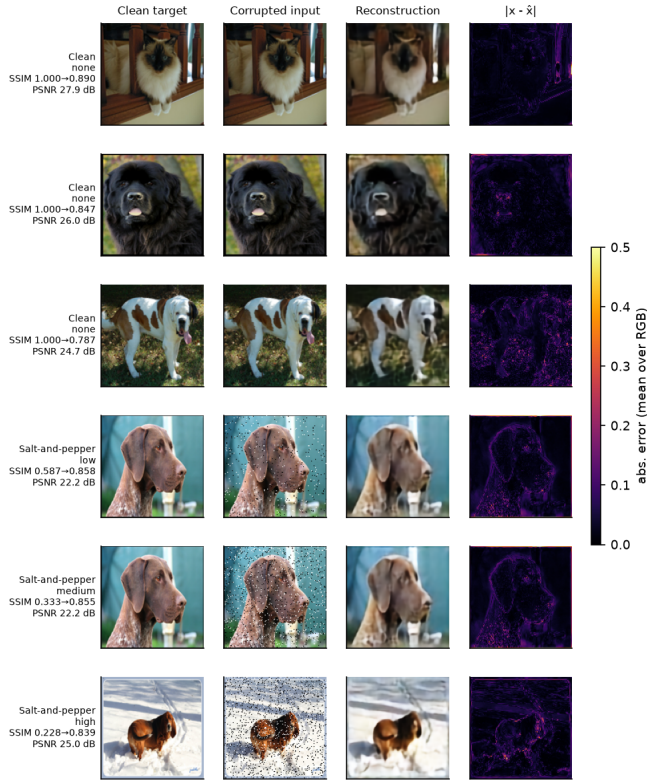


Fig. 4. Task 1 test SSIM of the corrupted input and the restored output per corruption and severity.

already clean, so it always re-encodes it. These failures motivate the identity branch of Tasks 2 and 3.

Task 1: representative test examples (median J per condition/level)



Task 1: representative test examples (median J per condition/level)

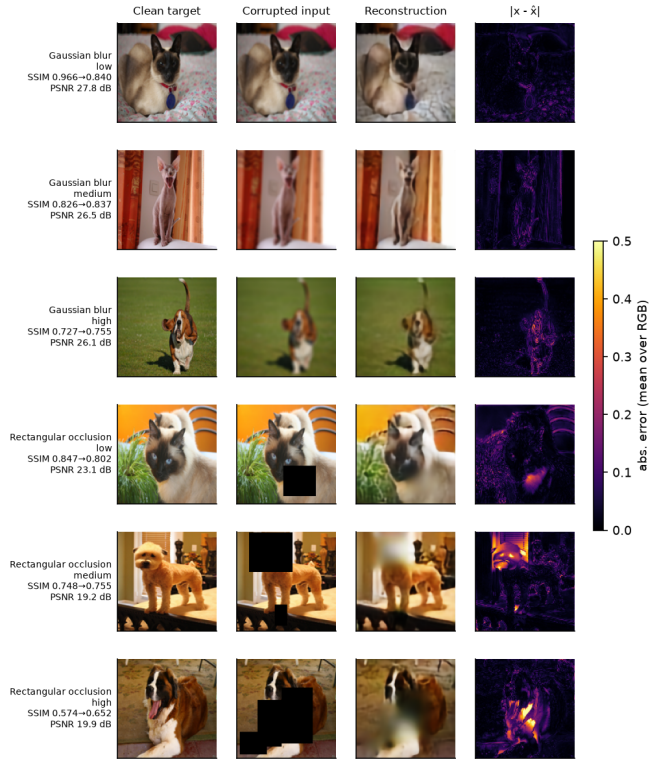


Fig. 5. Twelve representative test examples (the image closest to the median J of each condition and level; three for clean): clean target, corrupted input, reconstruction and absolute error map.

Task 1: failure cases

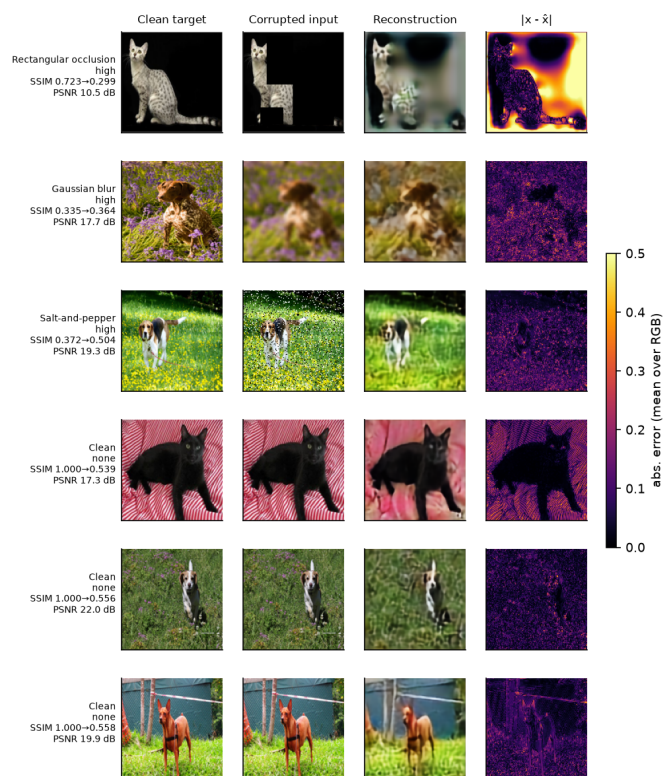


Fig. 6. Task 1 failure cases: the worst output per corruption type and the largest degradations of a clean input.

VI. Task 2: Classification and Hard-Routed Specialists

A. Corruption classifier

The classifier is a VGG-style CNN (stages of two convolution–BatchNorm–ReLU layers and max pooling). The first stage works at full resolution so that single-pixel salt-and-pepper evidence survives, and the pooled feature concatenates global average pooling (global blur statistics) with global max pooling (sparse, extreme evidence such as black rectangles). It is trained with cross-entropy and AdamW [35] on balanced batches; model selection uses 1–macro-F1 on the validation manifest. The Optuna study (25 trials of 10 epochs; 5 completed, 20 pruned) found a small model: three stages with 16/32/64 channels, dropout 0.18, weight decay $1.6 \cdot 10^{-3}$, learning rate $3.8 \cdot 10^{-4}$ and batch 32 (72.8k parameters). Dropout, weight decay and the learning rate share the importance (0.40/0.30/0.30), while the channel plan does not matter, so the task is easy enough for a small network. Most trials were pruned because the median pruner stops trials that are not better than earlier ones in the same epoch, which is common when all reasonable configurations reach >99 % accuracy. On the 36,690 test records the classifier reaches an accuracy of 99.42 %, with macro precision 0.987, recall 0.994 and F1 0.990. Salt-and-pepper is recognised perfectly. The remaining errors are all confusions with clean (Fig. 7): 0.57 % of clean images are predicted as blur and 0.27 % as occlusion, while 0.86 % of blurred and 0.80 % of occluded images are predicted as clean. All of them occur at the lowest severity: the accuracy is 97.4 % for mild blur ($k = 3$, $\sigma = 0.7$) and 97.8 % for 10 % occlusion, and ≥ 99.8 % everywhere else. Mildly blurred images of already soft photographs and small occluders on dark backgrounds are genuinely ambiguous.

B. Specialist autoencoders

The three specialists share one architecture found by a shared Optuna search, as permitted by the assignment: every trial trains all three specialists (each only on its own corruption) and minimises their mean validation J . Of 20 trials (11 completed, 9 pruned), the best reduced the mean J from 0.259 (baseline) to 0.174. The selected configuration has a $16 \times 16 \times 16$ bottleneck ($12 \times$ compression), $C = 32$, $\alpha = 0.52$ and learning rate $2.0 \cdot 10^{-3}$ (466k parameters per expert); the learning rate (importance 0.62) and α (0.28) mattered most. The three specialists were then trained independently for 60 epochs, reaching validation SSIM 0.824 (salt), 0.817 (blur) and 0.715 (occlusion).

C. Hard routing: oracle vs. predicted

In oracle mode the manifest label selects the expert; in predicted mode the classifier does. A clean route uses the identity bypass, so the output equals the input exactly (its PSNR is infinite and is capped at 100 dB in the tables; we therefore compare systems by SSIM). The two modes are almost identical (Fig. 8): the mean SSIM is 0.8018 with

TABLE V

Task 2 classifier Optuna study task2_classifier_final: 25 trials (pruned 20, complete 5); 10 epochs per trial; best objective 0.0054 (trial 22).

Parameter	Type	Search range	Best
lr	float	[0.0001, 0.003] (log)	0.0003755
batch_size	categorical	{32, 64, 128}	32
base_channels	categorical	{16, 32, 48}	16
channel_plan	categorical	{shallow, standard, wide_top}	shallow
dropout	float	[0, 0.5]	0.1811
weight_decay	float	[1e-06, 0.01] (log)	0.001638

TABLE VI

Task 2 classifier on the test manifest (accuracy 0.9942, macro-F1 0.9904).

Class	precision	recall	f1	support
clean	0.9521	0.9916	0.9714	3669
salt_pepper	1.0000	1.0000	1.0000	11007
blur	0.9981	0.9914	0.9947	11007
occlusion	0.9991	0.9920	0.9955	11007

oracle and 0.8023 with predicted routing, against 0.672 for the inputs. Predicted routing is even marginally better because some of its “errors” are beneficial: when a mildly blurred image is routed to the identity branch, the output keeps all of its detail, whereas the blur expert, limited by its bottleneck, lowers the SSIM of such near-clean inputs (oracle 0.825 vs. input 0.944). The comparison of oracle and predicted routing therefore shows that the classifier is not the bottleneck of the system; the restoration quality of the specialists is.

D. Failure analysis

Only 214 of 36,690 test records (0.58 %) are misrouted. We count a misrouting as a classifier-caused failure when the predicted route raises J by more than 0.01 compared with the oracle route; this happens 34 times (Fig. 9). Thirty-one of them are clean images sent to the blur (21) or occlusion (10) expert, which re-encode the image and destroy fine detail; the other three are mildly occluded images sent to the identity branch, which leaves the black rectangle unrepaired. Conversely, misrouting mild blur or mild occlusion to the identity branch lowered J on average (by 0.091 and 0.068), for the same reason as above. The most damaging error is therefore a clean image wrongly classified as corrupted.

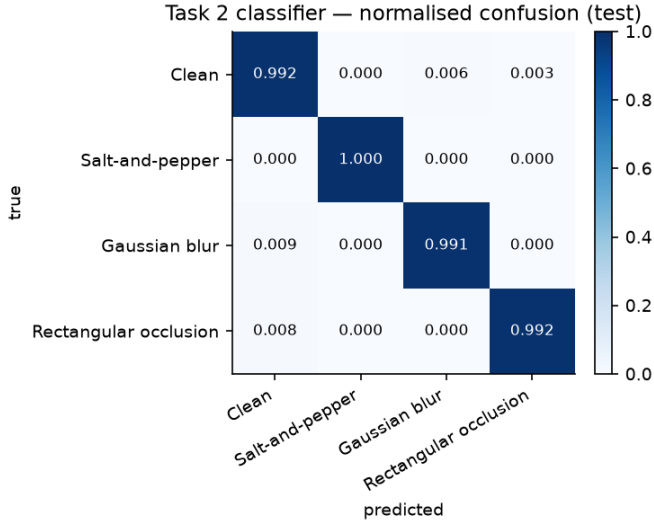


Fig. 7. Normalised four-class confusion matrix on the test manifest.

TABLE VII
Task 2 specialists (shared) Optuna study
task2_specialists_shared_final: 20 trials (complete 11, pruned 9);
10 epochs per trial; best objective 0.1743 (trial 16).

Parameter	Type	Search range	Best
lr	float	[0.0001, 0.003] (log)	0.002024
batch_size	categorical	{16, 32, 64}	32
bottleneck	categorical	{8x8x16, 8x8x32, 8x8x64, 16x16x8, 16x16x16}	16x16x16
base_channels	categorical	{16, 32, 48}	32
alpha	float	[0.5, 0.95]	0.5205

TABLE VIII
Task 2 hard routing on the test manifest: oracle vs. predicted routing.

Condition	oracle_psnr	pred_psnr	oracle_ssim	pred_ssim	input_ssim	cls_accuracy
clean / none	100.00	99.37	1.0000	0.9987	1.0000	0.9916
salt_pepper / low	25.31	25.31	0.8269	0.8269	0.6025	1.0000
salt_pepper / medium	25.29	25.29	0.8242	0.8242	0.3397	1.0000
salt_pepper / high	25.24	25.24	0.8185	0.8185	0.2041	1.0000
blur / low	25.59	25.74	0.8254	0.8294	0.9436	0.9741
blur / medium	25.75	25.75	0.8187	0.8187	0.8057	1.0000
blur / high	24.62	24.62	0.7463	0.7463	0.6916	1.0000
occlusion / low	22.70	22.76	0.7742	0.7765	0.8619	0.9779
occlusion / medium	21.32	21.32	0.7283	0.7283	0.7301	0.9981
occlusion / high	19.54	19.54	0.6556	0.6556	0.5458	1.0000

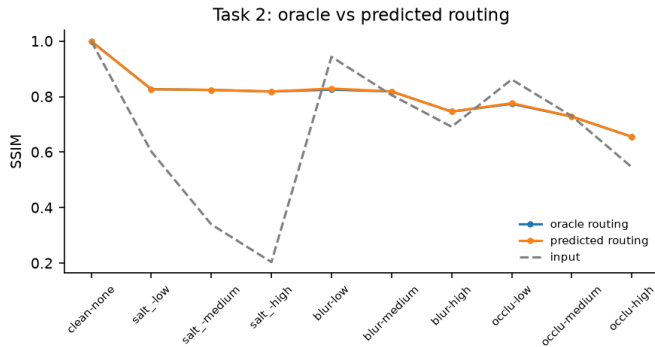


Fig. 8. Test SSIM of oracle and predicted routing per condition and severity.

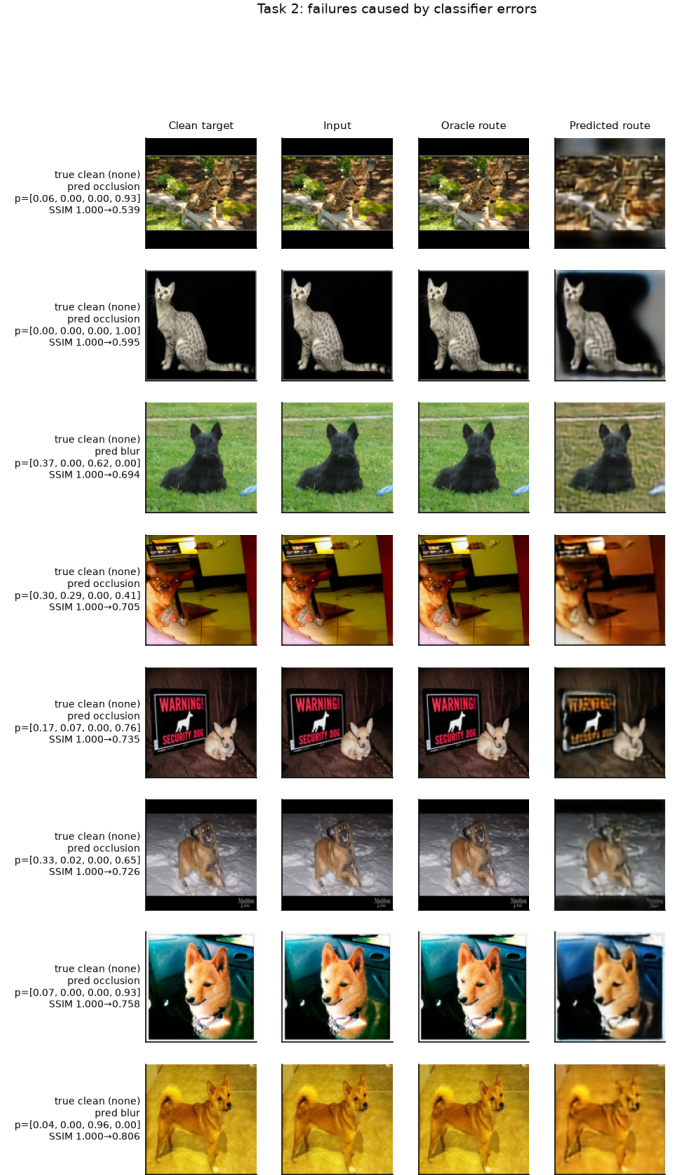


Fig. 9. Restoration failures caused by classifier errors: clean target, input, oracle route and predicted route.

VII. Task 3: Soft Mixture-of-Experts

A. Architecture and initialisation

The soft MoE (Fig. 10) computes $w = \text{softmax}(G(\tilde{x})/\tau)$ and $\hat{x} = w_0\tilde{x} + w_1A_{\text{salt}}(\tilde{x}) + w_2A_{\text{blur}}(\tilde{x}) + w_3A_{\text{occ}}(\tilde{x})$. Nothing is randomly initialised: the gate G is the trained Task 2 classifier (its class order equals the branch order) and the experts are the three trained specialists (1.47 M parameters in total). Evaluated without any further training, this combination already reaches validation SSIM 0.842.

B. Training strategy and joint objective

$\mathcal{L}_{\text{MoE}} = \lambda_1\mathcal{L}_1 + \lambda_s(1 - \text{SSIM}) + \lambda_c\mathcal{L}_{\text{CE}} + \lambda_b\mathcal{L}_{\text{balance}}$, with the cross-entropy applied to the untempered gate logits, so that τ only controls how sharply the gate’s belief is turned into weights. Training starts with a warm-up of three epochs in which the experts are frozen (including their BatchNorm statistics) and only the gate is trained with learning rate 10^{-4} ; then all parameters are fine-tuned for 15 epochs with the smaller learning rate selected by Optuna. Expert BatchNorm statistics remain frozen in the joint stage, because every expert now receives all four conditions, and updating its statistics with inputs it never specialised on would shift its normalisation.

Balance regulariser. The assignment’s loss $\mathcal{L}_{\text{balance}} = \sum_k (\bar{w}_k - 1/4)^2$ is computed over balanced batches. For a dense softmax the mean weights sum to one, so their mean is $1/4$ and Shazeer’s importance loss [11] $\text{CV}(\bar{w})^2 = 4 \sum_k (\bar{w}_k - 1/4)^2$ is the same loss up to a factor of four (verified by a unit test). As a research-supported alternative we implemented a marginal-entropy loss $\log 4 - H(\bar{w})$, which is also minimised by uniform usage but whose gradient grows as a branch dies out. The loss type is itself an Optuna parameter, and the final model was retrained with the other loss type as an ablation. The study (20 trials; 15 completed, 5 pruned, either as poor or because of routing collapse) improved the validation J from 0.093 for the assignment’s starting values ($\lambda_1 = 0.8$, $\lambda_s = 0.2$, $\lambda_c = 0.1$, $\lambda_b = 0.01$) to 0.082. The selected configuration uses $\tau = 2.08$, an equal weighting of L1 and SSIM ($\lambda_1 = 0.50$), a small classification weight $\lambda_c = 0.010$, a very small balance weight $\lambda_b = 1.9 \cdot 10^{-4}$ and a joint learning rate of $1.35 \cdot 10^{-5}$. The reconstruction weighting (importance 0.52) and the joint learning rate (0.24) mattered most; the balance type had no measurable influence. In the final run (Fig. 11) the warm-up alone improved the validation J from 0.096 to 0.088, and joint fine-tuning reduced it further to 0.080 (SSIM 0.869). Over the same time the fraction of inputs whose largest weight matches the true label fell from 0.994 to 0.898: the gate learned to deviate from pure classification where hedging gives a better reconstruction. The balance ablation with the entropy loss produced the same test SSIM (0.838), which is consistent with the tiny selected λ_b : on balanced batches with a well-initialised gate, routing collapse did not occur, and the balance term was not needed.

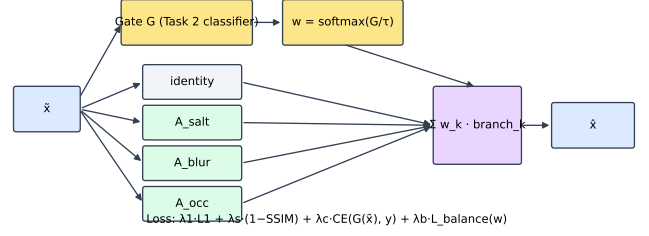


Fig. 10. Soft MoE: gate initialised from the Task 2 classifier, experts from the Task 2 specialists.

TABLE IX

Task 3 Optuna study task3_soft_moe_final: 20 trials (complete 15, pruned 5); 0 epochs per trial; best objective 0.0817 (trial 18).

Parameter	Type	Search range	Best
lr_joint	float	[1e-06, 5e-05] (log)	1.353e-05
tau	float	[0.5, 3]	2.077
ce_weight	float	[0.01, 0.5] (log)	0.01039
balance_weight	float	[0.0001, 0.1] (log)	0.0001863
recon_l1	float	[0.5, 0.95]	0.5009
balance	categorical	{squared, entropy}	squared

C. Routing analysis

The routing (Table X, Fig. 12) is sharp for clean (identity 0.92) and salt-and-pepper inputs (salt expert 0.90–0.99). For blur and occlusion the gate mixes the identity branch with the matching expert, and the mix depends on severity in a sensible way: the identity weight falls from 0.56 to 0.30 for blur and from 0.53 to 0.26 for occlusion as severity increases, while the expert weight rises from 0.38 to 0.64 (blur) and from 0.47 to 0.74 (occlusion). For mild corruption the input itself is a good estimate, so keeping part of it preserves detail that the bottleneck expert would lose. 34.6% of all test inputs are routed with a largest weight above 0.9 (Fig. 14) and 21.2% with a largest weight below 0.6 (Fig. 15); the mean routing entropy is 0.54 nats (maximum 1.39). No expert is inactive: the mean weights are 0.34 (identity), 0.29 (salt), 0.17 (blur) and 0.20 (occlusion), and every branch is the dominant branch for 21–30% of the inputs (Fig. 13). The only weight above 0.3 on an unrelated class is the identity branch on blur and occlusion inputs (0.39 on average), which, as explained above, is a deliberate hedge rather than a collapse. The salt and blur experts receive at most 0.05 weight on unrelated classes.

D. Results and comparison

The soft MoE is the best restoration system on the test set (overall SSIM 0.838, Fig. 16). It almost preserves clean images (SSIM 0.998, PSNR 49.5 dB; not exactly 1, because the gate never gives the identity exactly all of the weight), and it is the best system at every corruption level. The largest gains over hard routing are at low severity, where hedging with the identity pays off (mild blur: 0.913 vs.

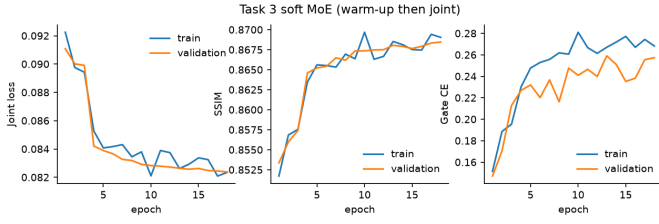


Fig. 11. Task 3 training: three warm-up epochs (frozen experts) followed by joint fine-tuning.

TABLE X

Average soft-MoE routing weight per true condition and severity (test).

Condition	Identity	Salt expert	Blur expert	Occlusion expert
clean / none	0.922	0.009	0.025	0.045
salt_pepper / low	0.081	0.901	0.001	0.017
salt_pepper / medium	0.013	0.982	0.001	0.004
salt_pepper / high	0.011	0.986	0.001	0.003
blur / low	0.556	0.019	0.376	0.049
blur / medium	0.331	0.019	0.609	0.040
blur / high	0.296	0.020	0.643	0.041
occlusion / low	0.525	0.002	0.003	0.469
occlusion / medium	0.388	0.002	0.002	0.608
occlusion / high	0.258	0.002	0.001	0.739

0.829; 10 % occlusion: 0.876 vs. 0.777); at high severity the systems converge (blur 0.761 vs. 0.746, occlusion 0.684 vs. 0.656), because then the experts' restoration ability limits all systems.

E. Failure analysis

The worst Task 3 outputs are the same hard images as in Task 1 (e.g. the cat on a black background with 35 % occlusion, SSIM 0.723→0.282): the occlusion expert, like the universal model, fills large holes with a smooth average colour. The four largest degradations are all medium or high occlusions of cats with dark or highly textured fur, where the inpainted region has the wrong colour. As an additional analysis, not part of the official protocol, we tested images with two corruptions, the situation that motivates soft routing. For blur+salt-and-pepper at medium severity the SSIM is 0.241 for the input, 0.737 with hard routing and 0.761 with the soft MoE; for occlusion+salt-and-pepper it is 0.252/0.602/0.609 and for blur+occlusion 0.588/0.668/0.681. The soft MoE is better in every combination, but the gains are small: because training contained only single corruptions, no expert has learned to remove two corruptions at once.

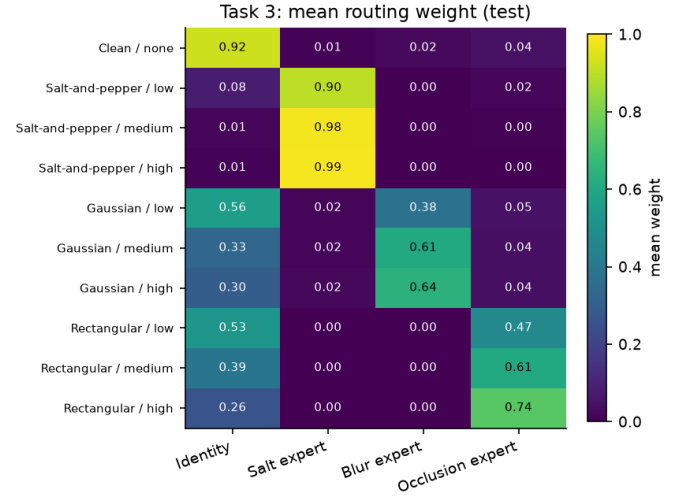


Fig. 12. Average routing weight per true condition and severity (test).

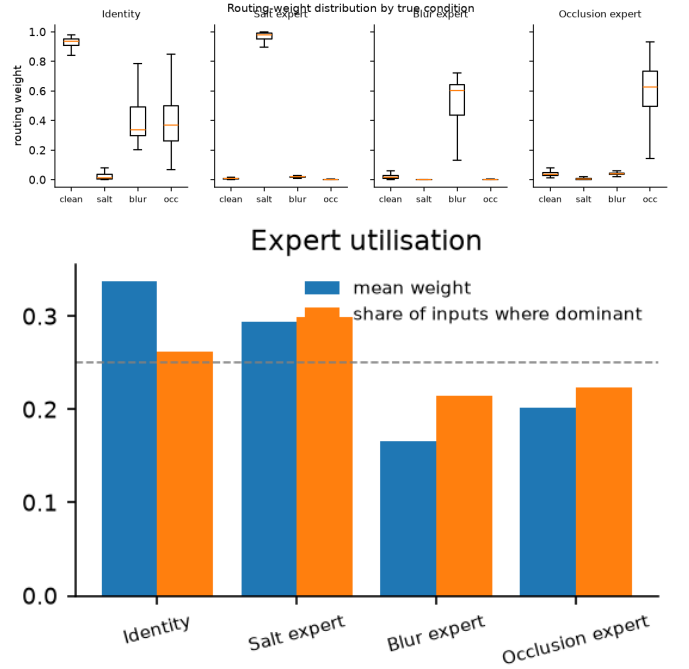


Fig. 13. Routing-weight distributions per true condition and expert utilisation.

Task 3: one expert dominates (max weight > 0.90)

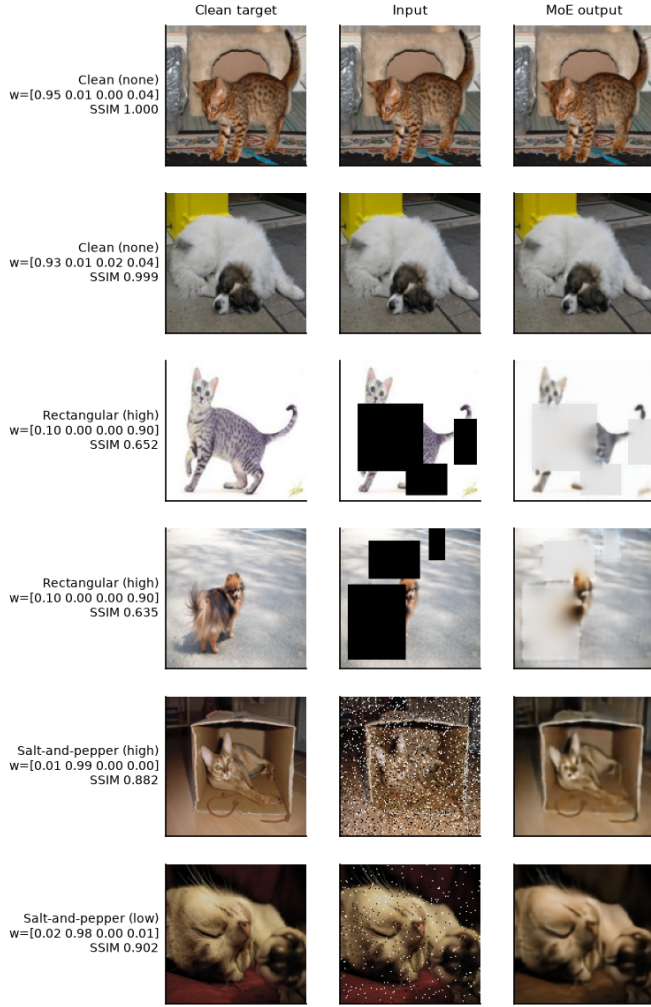


Fig. 14. Examples in which one branch dominates (largest weight > 0.9).

TABLE XI
Task 3 soft MoE test results.

Condition	l1	psnr	ssim	input_psnr	input_ssime	n
clean / none	0.0025	49.53	0.9984	100.00	1.0000	3669
salt_pepper / low	0.0334	26.69	0.8489	20.13	0.6025	3669
salt_pepper / medium	0.0366	26.13	0.8373	15.88	0.3397	3669
salt_pepper / high	0.0373	25.96	0.8295	13.14	0.2041	3669
blur / low	0.0220	29.93	0.9129	32.34	0.9436	3669
blur / medium	0.0303	27.38	0.8430	26.77	0.8057	3669
blur / high	0.0365	25.65	0.7613	24.35	0.6916	3669
occlusion / low	0.0333	23.12	0.8763	16.69	0.8619	3669
occlusion / medium	0.0458	21.70	0.7925	13.36	0.7301	3669
occlusion / high	0.0635	19.58	0.6838	10.79	0.5458	3669

Task 3: weights distributed over several branches

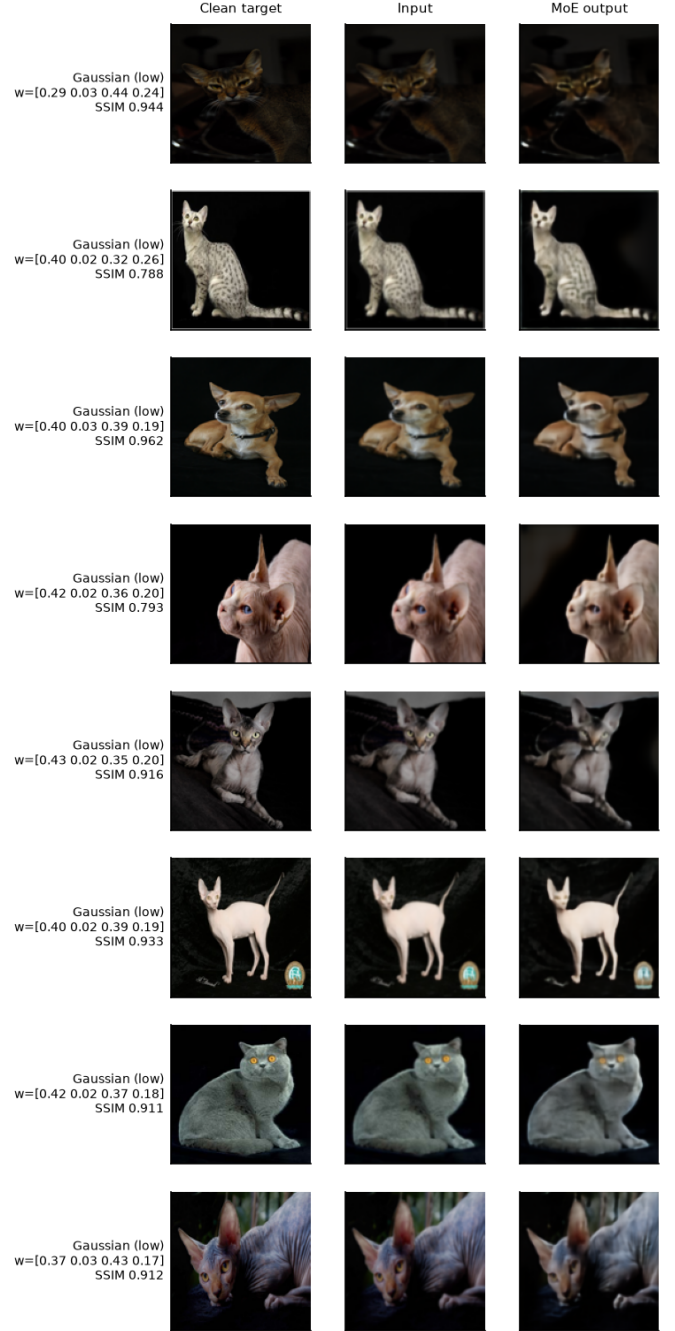


Fig. 15. Examples with weights distributed over several branches (largest weight < 0.6).

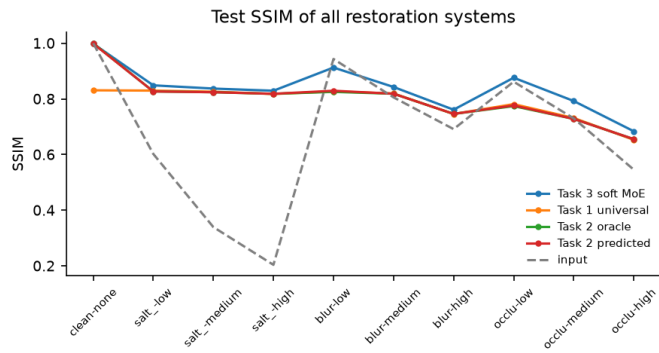


Fig. 16. Test SSIM of Task 1, Task 2 (oracle/predicted) and Task 3 per condition and severity.

VIII. Task 4: Style-Conditioned Face-to-Sketch cGAN

A. Dataset

See Section III-C: the 899/159 training/validation pairs are used for training and model selection; the official test set (1,046 pairs) only for the final evaluation.

B. cGAN architecture and style conditioning

The generator (Fig. 17) is a pix2pix U-Net adapted to 128×128 inputs: seven stride-2 downsampling stages (to 1×1) and a mirrored decoder with skip connections, InstanceNorm [36] and dropout in the first three decoder blocks (42.0 M parameters). The discriminator is a 70×70 PatchGAN that classifies 14×14 overlapping patches of a (photo, sketch) pair (2.8 M parameters). Each network has its own learned embedding $e_s \in \mathbb{R}^{16}$ for the three FS2K styles. We compared three ways of conditioning: spatial replication and concatenation [22], conditional normalisation / FiLM [23], [24], and a projection discriminator [25]. We chose concatenation because it conditions both networks with very little code and exports directly to ONNX. To prevent the style signal from fading in deep layers, the generator receives the embedding at the input (tiled to 128×128) and again at the 1×1 bottleneck; the discriminator receives it tiled together with the photo and the sketch. A unit test confirms that changing the style conditions changes the outputs of both networks.

C. Losses

$\mathcal{L}_G = \text{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1} \|y - G(x, s)\|_1$ and $\mathcal{L}_D = \frac{1}{2}[\text{BCE}(D(x, y, s), 1) + \text{BCE}(D(x, G(x, s), s), 0)]$, with binary cross-entropy on logits, Adam ($\beta_1 = 0.5$) [37] and a linear learning-rate decay over the second half of training, as in pix2pix [20].

D. Training

Optuna trials used 25-epoch schedules. 22 trials were started: 18 completed, 2 were pruned and 2 were interrupted when the training process was stopped externally and never finished; they are excluded. The pix2pix defaults (trial 0: learning rates $2 \cdot 10^{-4}$, $\lambda_{L1} = 100$, dropout 0.5) reached $J = 0.327$, the best trial 0.313. The selected configuration uses $\lambda_{L1} = 148.7$, dropout 0.12, a generator learning rate of $4.5 \cdot 10^{-4}$, a six times smaller discriminator learning rate of $7.4 \cdot 10^{-5}$, batch 8, 64 base channels and a 16-dimensional style embedding. λ_{L1} dominated the importance (0.67). Because the selection objective is pixel-based, it favours a large λ_{L1} and therefore conservative, slightly smoother sketches; this is a known limitation of selecting GANs with paired pixel metrics. The selected configuration was retrained for the full 200 epochs, and the checkpoint with the best validation J (epoch 113) is deployed.

The losses were logged separately (Fig. 18). Training was stable: the discriminator’s mean probability stayed around 0.72 for real and 0.28 for generated pairs from epoch 25 to 200, so neither network overpowered the other and no

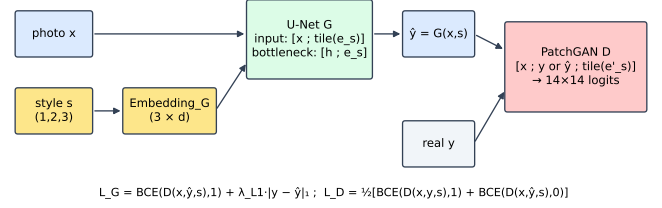


Fig. 17. U-Net generator and PatchGAN discriminator, each with its own learned style embedding.

TABLE XII

Task 4 Optuna study task4_pix2pix_final: 22 trials (complete 18, running 2, pruned 2); 25 epochs per trial; best objective 0.3133 (trial 19).

Parameter	Type	Search range	Best
lr_g	float	[5e-05, 0.0005] (log)	0.000452
lr_d	float	[5e-05, 0.0005] (log)	7.378e-05
batch_size	categorical	{4, 8, 16}	8
base_channels	categorical	{32, 48, 64}	64
dropout	float	[0, 0.5]	0.1172
style_dim	categorical	{4, 8, 16, 32}	16
lambda_l1	float	[10, 200] (log)	148.7

mode collapse occurred. The generator’s training L1 kept decreasing (0.284→0.171) while the validation L1 stayed at 0.102–0.103 after epoch 25, so later epochs mainly improved texture on the training images. Fig. 19 shows how the outputs on the same validation photos develop from blurry outlines into stroke-based sketches.

E. Results

The assignment-required reconstruction measure is the paired L1 error between the generated and the true sketch; SSIM and PSNR are reported as additional, research-based measures of structural and pixel fidelity. We did not use FID: with about 1,000 test images it is strongly biased [27], and its Inception features are not designed for grayscale line drawings. On the official test set the generator reaches L1 0.111, SSIM 0.462 and PSNR 15.05 dB (validation: L1 0.102, SSIM 0.482). Style 2 is clearly the hardest (SSIM 0.372 vs. 0.507 for Style 1 and 0.606 for Style 3): its sketches are drawn with dark, dense strokes whose exact placement varies between drawings, which pixel-wise metrics penalise heavily. Because the test set contains 619 Style 1, 381 Style 2 and only 46 Style 3 images, the overall mean is dominated by Styles 1 and 2, and the Style 3 result rests on few images. The style embedding demonstrably controls the output: generating the same test photo under two different styles changes the sketch by 0.116 (Style 1 vs. 2), 0.115 (1 vs. 3) and 0.097 (2 vs. 3) mean absolute intensity, and Fig. 20 shows the visible change from light outlines (Style 1) to dark shaded strokes (Style 2).

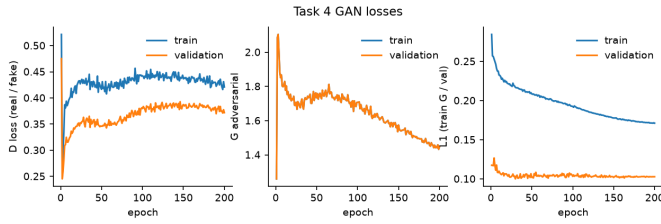


Fig. 18. Discriminator real/fake loss, generator adversarial loss and L1 loss during the final training.

TABLE XIII

Task 4 generator on the FS2K test split. L1 is the assignment-required reconstruction measure; SSIM/PSNR (and LPIPS if shown) are additional.

Style	l1	ssim	psnr	n
1	0.0826	0.5073	16.83	619
2	0.1624	0.3723	11.81	381
3	0.0707	0.6061	17.86	46
overall	0.1111	0.4625	15.05	1046

F. Failure analysis

The worst test results are all Style 2 sketches of photo1 faces (SSIM 0.13–0.18, Fig. 21). In these cases the generator places dark hair and shading strokes where the artist drew few lines, or the other way round; the sketch is plausible but differs from the particular drawing. Because style is confounded with the photo source in FS2K, the embedding has partly learned the drawing habits of each source, which is visible when a photo is rendered in a style that never occurred with its source. Finally, at 128×128 pixels thin strokes are only one or two pixels wide, so small misalignments already produce large L1 errors.

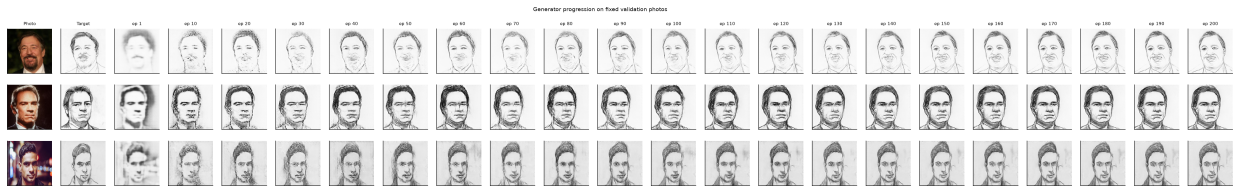


Fig. 19. Generator outputs on the same fixed validation photos across the 200 training epochs.

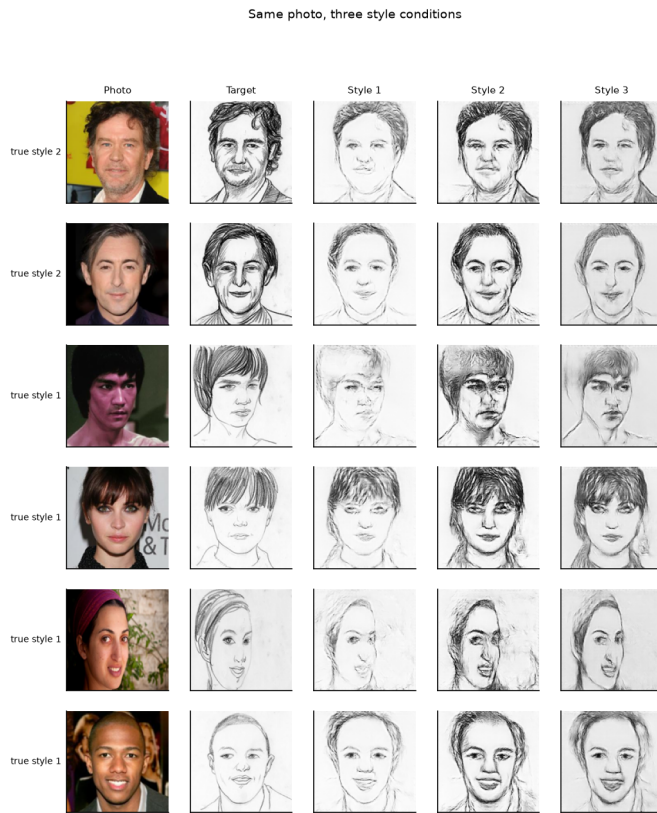


Fig. 20. The same photograph under the three style conditions (test set).

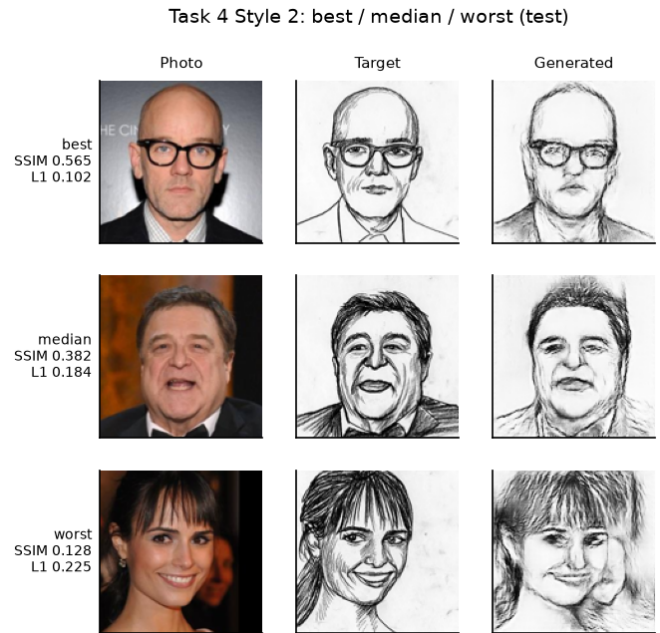


Fig. 21. Style 2 test examples: best, median and worst by SSIM.

IX. Application Architecture

All four systems are served by one web application (Fig. 22). The React/Tailwind frontend is built and served by nginx, which forwards /api requests to the backend. The FastAPI backend validates uploads by content (format, size limit, image dimensions), converts them to RGB and resizes them to 128×128 , optionally applies a corruption at runtime with exactly the same code that generated the training data, runs the ONNX models and returns the images, probabilities, routing weights and timings as JSON. It provides health-check, universal-restoration, hard-routing, soft-mixture and face-to-sketch endpoints. The backend contains no PyTorch: it only needs ONNX Runtime, NumPy and Pillow, which a test verifies. The four workspaces show the input, the restored or generated image, the corruption settings and the inference time; the Hard-Routed workspace adds the four class probabilities and the selected expert, the Soft MoE workspace the four routing weights with a stacked bar, and the Face-to-Sketch workspace supports upload, webcam capture, style selection and download (Figs. 24–27).

A. Interface design in Google Stitch

The interface was first designed in Google Stitch (Figs. 25–26): a fixed sidebar listing the four workspaces and the system status page, a header bar, white rounded cards on a light canvas, an indigo accent colour, monospace labels for technical values, and a two-column workspace layout (input and corruption controls on the left; an “inspection stage” with tagged image panels, analysis and telemetry cards on the right). These design tokens and components were transferred to the React/Tailwind implementation (frontend/src/index.css, components/ui.jsx, workspaces/*.jsx); Fig. 27 shows the implemented application. Note: the Stitch mock-ups contain placeholder content generated by Stitch (e.g. face photographs in Task 1, hardware names, model names and metric values); these are not results of this project. Only the visual design was adopted; every value shown by the implemented application comes from the backend. The main deviations from the mock-ups follow from this: images are shown at the models’ 128×128 resolution, controls that Stitch invented without a model behind them (e.g. a stroke-density slider or SVG export) were omitted, and the style cards show the three FS2K style categories.

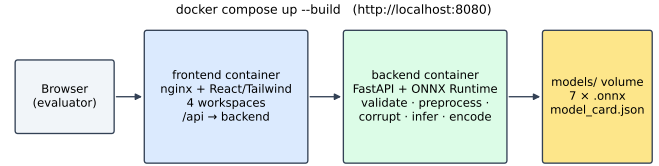


Fig. 22. Deployment: React/Tailwind frontend (nginx) and FastAPI + ONNX Runtime backend in Docker Compose.

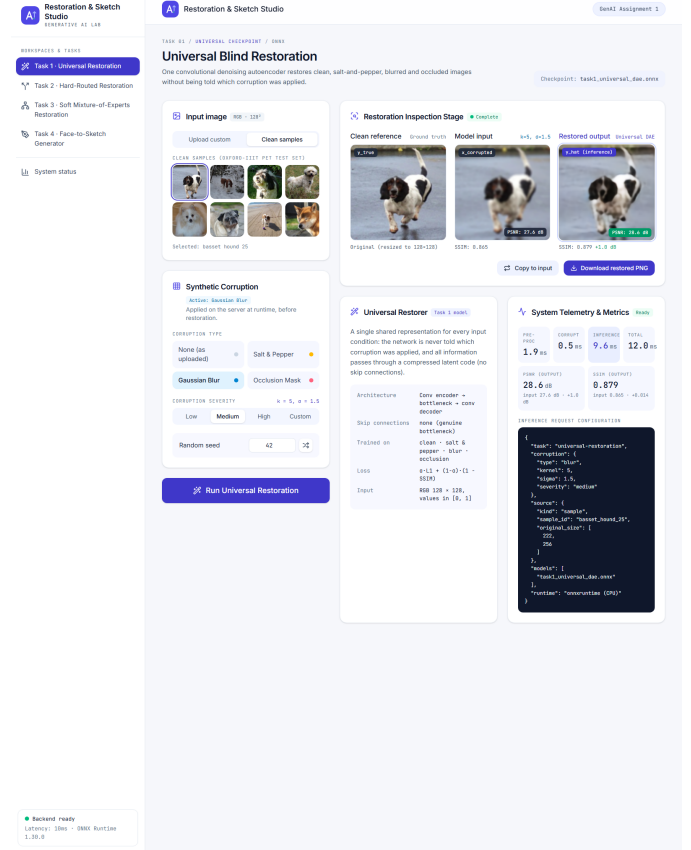


Fig. 23. Universal Restoration workspace with the deployed model.

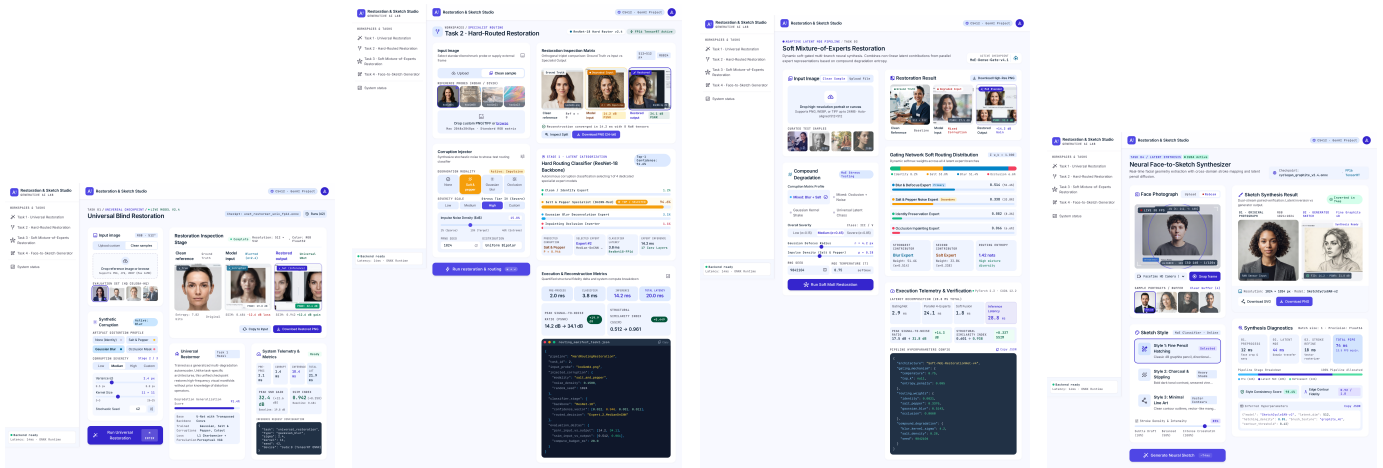


Fig. 25. Original Google Stitch designs of the four workspaces: Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts Restoration and Face-to-Sketch Generator (placeholder content generated by Stitch).

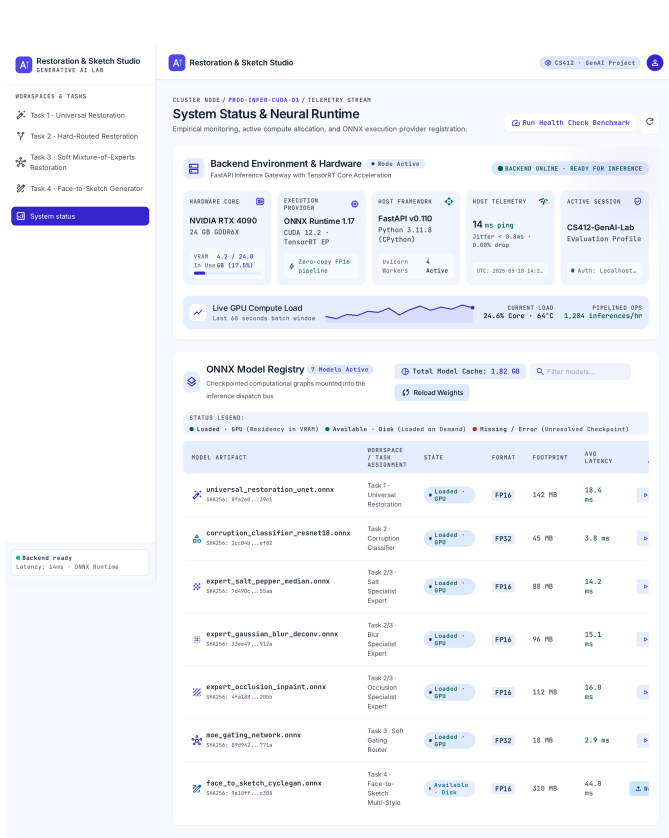


Fig. 26. Google Stitch design of the System status page.

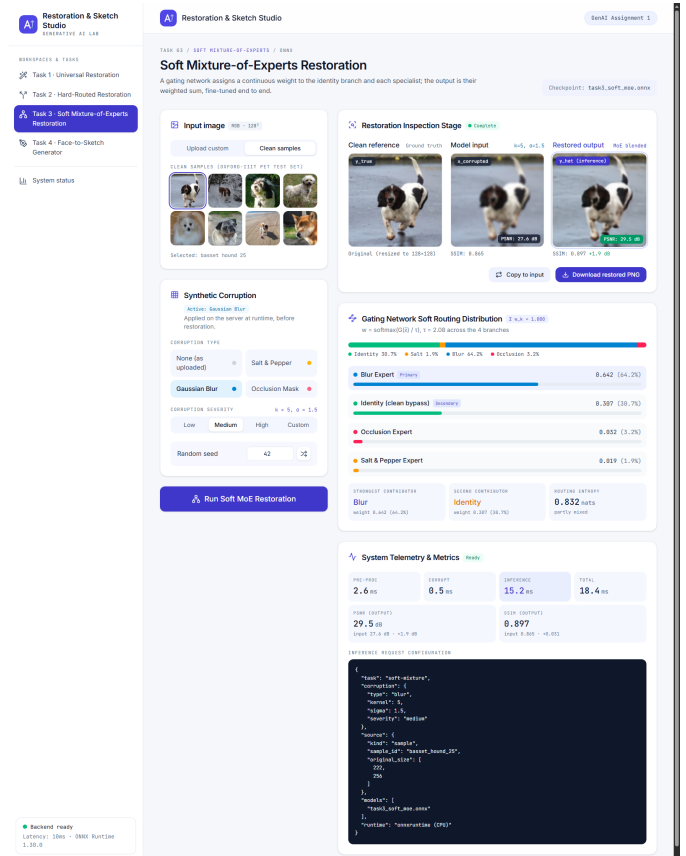


Fig. 27. Implemented application (Soft MoE workspace) following the Stitch design.

X. ONNX Deployment

All seven deployed models were exported with the torch.export-based ONNX exporter (opset 18) with a dynamic batch dimension. Each export was validated by running the same inputs (real validation images and random tensors, at a batch size different from the export batch) through PyTorch and ONNX Runtime and comparing shapes and values; every output passed, with a maximum absolute difference of $8.1 \cdot 10^{-6}$, far below one 8-bit grey level ($3.9 \cdot 10^{-3}$) (Table XIV). The complete soft MoE (gate, softmax with the learned τ , three experts and the weighted sum) is a single static graph with two outputs, so it exports without any architectural change. Hard routing contains data-dependent control flow, so it is deployed as four graphs, with the argmax and the identity bypass in the backend; this also means that no expert is executed for inputs predicted as clean. The discriminator is not exported.

XI. Docker and Reproducibility

The frontend and backend run in two containers started with one command, docker compose up --build; the application is then available at <http://localhost:8080>. The ONNX models are not stored in the Git repository but attached to a GitHub release and mounted read-only into the backend container; the backend has a health check, and the frontend container starts only when the backend is healthy. Ports, the model path and the upload limit are configurable through environment variables. Reproducibility of the experiments rests on fixed seeds, the committed split and corruption manifests, the committed Optuna studies, pinned dependency versions and one driver script (scripts/run_pipeline.py) that runs every study, training, evaluation, export and figure step; a SMOKE mode runs the entire pipeline on a tiny subset in a few minutes, and 67 automated tests cover the corruptions, manifests, models, losses, ONNX consistency and API.

XII. Limitations

All models work at 128×128 pixels, which limits detail in both restoration and sketches. The genuine bottleneck costs fidelity on clean and mildly corrupted images (Task 1 clean SSIM 0.831); the limited-skip ablation shows how much is lost. Training contained only one corruption per image, so no system removes combined corruptions well. Our selection objectives are pixel- and structure-based, which favours smooth outputs, particularly for the GAN. FS2K style is confounded with the photo source and the Style 3 test set is small, so the per-style results must be interpreted with care. Bit-exact GPU reproducibility is not guaranteed by PyTorch, so repeated runs can differ slightly. Finally, the Stitch mock-ups contain placeholder content and served only as a visual design.

TABLE XIV
PyTorch vs. ONNX Runtime on real validation inputs plus random inputs (tolerance: np.allclose, atol 10^{-4} , rtol 10^{-3}).

Model	Output	Max abs. diff.	Mean abs. diff.	Pass
Task 1 universal DAE	restored	$2.8 \cdot 10^{-6}$	$9.7 \cdot 10^{-8}$	✓
Task 2 classifier	logits	$3.8 \cdot 10^{-6}$	$7.6 \cdot 10^{-7}$	✓
Task 2 salt expert	restored	$2.9 \cdot 10^{-6}$	$1.1 \cdot 10^{-7}$	✓
Task 2 blur expert	restored	$3.5 \cdot 10^{-6}$	$1.3 \cdot 10^{-7}$	✓
Task 2 occlusion expert	restored	$7.6 \cdot 10^{-6}$	$1.9 \cdot 10^{-7}$	✓
Task 3 soft MoE	restored	$3.2 \cdot 10^{-6}$	$1.0 \cdot 10^{-7}$	✓
Task 3 soft MoE	weights	$2.1 \cdot 10^{-7}$	$3.4 \cdot 10^{-8}$	✓
Task 4 generator	sketch	$8.1 \cdot 10^{-6}$	$3.1 \cdot 10^{-7}$	✓

XIII. Conclusion

For blind multi-corruption restoration with a genuine bottleneck, routing to specialists is clearly better than a single universal model, and soft routing is better than hard routing: the test SSIM increases from 0.786 (universal) to 0.802 (hard) and 0.838 (soft). The decisive advantage of both routed systems is the identity branch, which keeps clean inputs intact. The hard router’s classifier is nearly perfect (99.42 %), so its errors hardly affect the overall quality; the soft MoE’s advantage comes from hedging between the identity and an expert for mild corruptions, a behaviour that emerged from joint training although the gate was initialised as a classifier. The balance loss turned out to be unnecessary on balanced batches with a well-initialised gate. For face-to-sketch synthesis, a single pix2pix model with a learned style embedding in both networks produces three distinguishable styles, with Style 2 the hardest to reproduce. Higher resolution, training with combined corruptions and perceptual selection criteria are the most promising next steps.

Appendix AI Use

AI assistants were used during this project, and all their output was checked as described here. Claude Code (Anthropic) was used to extract the requirement checklist from the assignment PDF, to plan the repository, to implement and debug the data pipeline, models, training, evaluation, ONNX export, FastAPI backend and React frontend, to write automated tests and documentation, and to find and verify literature (every reference was checked against its DOI, arXiv or proceedings page). Google Stitch generated the interface mock-ups. Verification: 67 unit and API tests (e.g. corruption ranges, manifest determinism, SSIM against scikit-image, paired-augmentation alignment, MoE weights summing to one, style influence on both networks); a SMOKE run of the complete pipeline before the final experiments; inspection of training curves, logs and generated figures; numerical PyTorch–ONNX comparison of every deployed model; and manual testing of the application in the browser and in Docker. Problems found and corrected this way included an incorrect floating-point comparison in a test, a pandas

incompatibility in the Task 3 evaluation, an insufficient Task 1 search space (replaced by a bottleneck-shape search after the first trial) and an unreliable dependency resolution in the Docker build.

References

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in Proc. 25th Int. Conf. Machine Learning (ICML), 2008, pp. 1096–1103.
- [2] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *Journal of Machine Learning Research*, vol. 11, pp. 3371–3408, 2010. [Online]. Available: <https://jmlr.org/papers/v11/vincent10a.html>
- [3] X.-J. Mao, C. Shen, and Y.-B. Yang, "Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections," in *Advances in Neural Information Processing Systems 29 (NIPS)*, 2016.
- [4] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [5] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," *IEEE Transactions on Computational Imaging*, vol. 3, no. 1, pp. 47–57, 2017.
- [6] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [7] A. Horé and D. Ziou, "Image quality metrics: PSNR vs. SSIM," in Proc. 20th Int. Conf. Pattern Recognition (ICPR), 2010, pp. 2366–2369.
- [8] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2536–2544.
- [9] A. Odena, V. Dumoulin, and C. Olah, "Deconvolution and checkerboard artifacts," *Distill*, 2016. [Online]. Available: <https://distill.pub/2016/deconv-checkerboard>
- [10] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [11] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.
- [12] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–40, 2022. [Online]. Available: <https://jmlr.org/papers/v23/fedus21-0998.html>
- [13] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations (ICLR)*, 2021.
- [14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015.
- [15] G. Pereyra, G. Tucker, J. Chorowski, L. Kaiser, and G. Hinton, "Regularizing neural networks by penalizing confident output distributions," 2017.
- [16] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems 27 (NIPS)*, 2014, pp. 2672–2680.
- [17] M. Mirza and S. Osindero, "Conditional generative adversarial nets," 2014.
- [18] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *International Conference on Learning Representations (ICLR)*, 2016.
- [19] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANs," in *Advances in Neural Information Processing Systems 29 (NIPS)*, 2016, pp. 2226–2234.
- [20] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017, pp. 5967–5976.
- [21] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. LNCS, vol. 9351. Springer, 2015, pp. 234–241.
- [22] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, "StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 8789–8797.
- [23] H. de Vries, F. Strub, J. Mary, H. Larochelle, O. Pietquin, and A. Courville, "Modulating early visual processing by language," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.
- [24] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in Proc. AAAI Conf. Artificial Intelligence, vol. 32, no. 1, 2018.
- [25] T. Miyato and M. Koyama, "cGANs with projection discriminator," in *International Conference on Learning Representations (ICLR)*, 2018.
- [26] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 586–595.
- [27] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.
- [28] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD), 2019.
- [29] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems 24 (NIPS)*, 2011, pp. 2546–2554.
- [30] K. Jamieson and A. Talwalkar, "Non-stochastic best arm identification and hyperparameter optimization," in Proc. 19th Int. Conf. Artificial Intelligence and Statistics (AISTATS), ser. PMLR, vol. 51, 2016, pp. 240–248. [Online]. Available: <https://proceedings.mlr.press/v51/jamieson16.html>
- [31] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018. [Online]. Available: <https://jmlr.org/papers/v18/li16-558.html>
- [32] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2012, pp. 3498–3505.
- [33] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. Van Gool, "Facial-sketch synthesis: A new challenge," *Machine Intelligence Research*, vol. 19, no. 4, pp. 257–287, 2022.
- [34] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd Int. Conf. Machine Learning (ICML), ser. PMLR, vol. 37, 2015, pp. 448–456. [Online]. Available: <https://proceedings.mlr.press/v37/ioffe15.html>
- [35] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.
- [36] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," 2016.
- [37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.